SerpApi India Hackathon 2026 — AI Agents Track: Full Build Guide

Track: 01 — AI Agents (Recommended Tooling) **Goal:** A solo-buildable agent that searches, compares, and synthesizes a cited answer using live SerpApi data. **Deadline:** Oct 5, 2026, 23:59 IST

This guide explains *what* to build and *why*, so you understand every moving part instead of copy-pasting a black box. Every section has a "Concept" callout before the code.

---

1. The core idea

Project: "Ask & Verify" — a research agent

User asks a question → the agent decides which SerpApi engine(s) to query → it fetches live results → it reads/compares them → it returns a synthesized answer **with citations back to real URLs**.

Example: *"What's the current cheapest flight from Pune to Delhi this weekend, and is there any news affecting flight prices?"* → Agent calls Google Flights engine + Google News engine → cross-references → answers with sources.

This is deliberately scoped to **2 SerpApi engines** so you can finish, polish, and record a clean demo — judges reward finished + well-demoed over ambitious + broken.

---

2. Concepts you're using (so you're not blind)

2.1 What "an agent" actually means here

> **Concept:** An LLM agent is just an LLM given (a) a set of callable tools with descriptions, and (b) a loop that lets it call a tool, read the result, and decide whether to call another tool or answer. It is *not* magic — it's a `while` loop plus structured function-calling.

The loop looks like:

```
1. Send user question + tool definitions to the LLM
2. LLM responds with either: (a) a final answer, or (b) a request to call tool X with args Y
3. If (b): you run tool X, feed the result back to the LLM, go to step 2
4. If (a): stop, return the answer
```

2.2 Tool-calling / function-calling

> **Concept:** Modern LLM APIs (Claude, OpenAI, etc.) support "tools" — you describe a function's name, purpose, and parameters in JSON schema. The model decides *when* and *with what arguments* to call it. You are still the one who executes the actual code; the model only decides intent.

This is different from RAG (below) — tool-calling is about *taking actions to get fresh data*, not retrieving from a fixed pre-indexed store.

2.3 RAG (Retrieval-Augmented Generation) — and why this project is "search-based RAG"

> **Concept:** RAG means: retrieve relevant documents/data first, then generate an answer grounded in that retrieved content, instead of relying purely on the model's training data. Classic RAG retrieves from a vector database of your own documents. Your project does **search-based RAG**: instead of a static vector DB, you retrieve from live web search (SerpApi) — which is actually a *stronger* pitch for a search-API hackathon because it shows real-time grounding, not stale embeddings.

2.4 MCP (Model Context Protocol) vs. direct SDK calls

> **Concept:** MCP is a standardized protocol that lets any MCP-compatible client (Claude Desktop, agent frameworks) discover and call tools exposed by an "MCP server" without custom integration code per tool. SerpApi's MCP server exposes their search engines as MCP tools.
>
> - **Use MCP** if you want your agent to be usable inside Claude Desktop/other MCP clients directly, or want to avoid writing your own tool-schema boilerplate.
> - **Use the direct Python SDK** / `serpapi-search-tools` if you're building a standalone Python app with a custom LLM loop (e.g., using the Anthropic or OpenAI API directly) — you get more control over exactly what happens each step, which is better for a hackathon demo where you want to *narrate* what the agent is doing.
>
> **This guide uses the direct SDK approach**, because it's easier to add visible logging/narration for your demo video, and doesn't require the judge to have an MCP client installed to see it work.

2.5 Multi-agent vs. single-agent

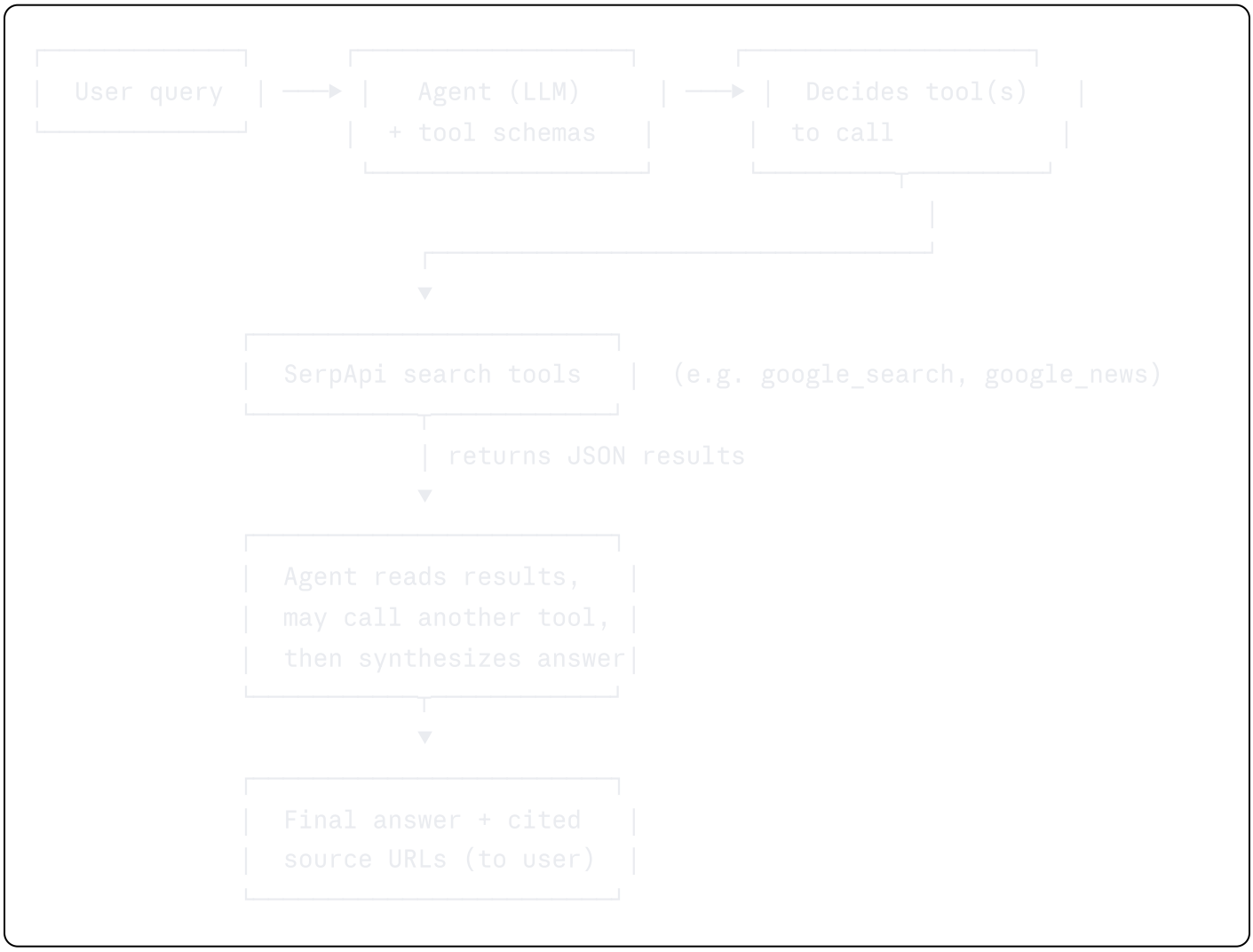
> **Concept:** "Multi-agent" means multiple LLM instances with different roles (e.g., a "planner" agent and a "verifier" agent) talking to each other. It sounds impressive but adds complexity and failure points. For a solo hackathon build, a **single agent with multiple tools** is the right scope — you get credit for the "multi-agent" tag in spirit (the agent orchestrates multiple search tools) without the coordination overhead.

2.6 Grounding & citation

> **Concept:** "Grounding" means every claim the model makes traces back to a specific retrieved source, not the model's memory. You enforce this by instructing the model (in its system prompt) to only state facts present in tool results, and to tag each claim with the source URL. This is the single most important design choice for how "trustworthy" your demo looks to judges.

---

3. Architecture



---

4. Project structure

```
serpapi-research-agent/
├── README.md
├── requirements.txt
├── .env.example
├── src/
│   ├── agent.py          # the agent loop (tool-calling logic)
│   ├── tools.py          # SerpApi tool wrappers + schemas
│   ├── prompts.py        # system prompt(s)
│   └── main.py           # CLI entry point
├── demo/
│   └── sample_queries.md # example questions to show in your demo video
└── tests/
    └── test_tools.py
```

---

5. Step-by-step build

Step 1 — Setup

bash

```
mkdir serpapi-research-agent && cd serpapi-research-agent
python -m venv venv && source venv/bin/activate
pip install serpapi-search-tools anthropic python-dotenv
```

Get a SerpApi key from the dashboard, and an Anthropic API key. Put both in `.env`:

```
SERPAPI_API_KEY=your_key_here
ANTHROPIC_API_KEY=your_key_here
```

> **Concept — why `.env`:** Never hardcode API keys in source files, especially since your repo must be **public** for submission. `.env` is loaded at runtime and excluded from git via `.gitignore`.

Step 2 — Define your tools (`src/tools.py`)

> **Concept:** Each tool needs (a) a JSON schema the model sees to know it exists and what args it takes, and (b) the actual Python function that executes it. Keep the description precise — vague tool descriptions are the #1 cause of an agent calling the wrong tool.

python

```python
import os
from serpapi import GoogleSearch  # or the specific serpapi-search-tools import

def web_search(query: str) -> dict:
    """Run a general web search and return top results with titles, links, snippet
    params = {
        "engine": "google",
        "q": query,
        "api_key": os.environ["SERPAPI_API_KEY"],
    }
    results = GoogleSearch(params).get_dict()
    return {
        "results": [
            {"title": r.get("title"), "link": r.get("link"), "snippet": r.get("sni
            for r in results.get("organic_results", [])[:5]
        ]
    }

def news_search(query: str) -> dict:
    """Run a news search and return recent articles with titles, links, dates."""
    params = {
        "engine": "google_news",
        "q": query,
        "api_key": os.environ["SERPAPI_API_KEY"],
    }
    results = GoogleSearch(params).get_dict()
    return {
        "results": [
            {"title": r.get("title"), "link": r.get("link"), "date": r.get("date")
            for r in results.get("news_results", [])[:5]
        ]
    }

# Tool schemas the LLM sees
TOOL_SCHEMAS = [
    {
        "name": "web_search",
        "description": "Search the general web for up-to-date information on any t
        "input_schema": {
            "type": "object",
            "properties": {"query": {"type": "string", "description": "The search
```

```
            "required": ["query"],
        },
    },
    {
        "name": "news_search",
        "description": "Search recent news articles related to a topic.",
        "input_schema": {
            "type": "object",
            "properties": {"query": {"type": "string", "description": "The news se
            "required": ["query"],
        },
    },
]

TOOL_FUNCTIONS = {"web_search": web_search, "news_search": news_search}
```

Step 3 — System prompt (`src/prompts.py`)

> **Concept — grounding via prompt:** You explicitly tell the model to (a) use tools before answering, (b) never state facts not present in tool results, (c) cite sources. This is your main lever against hallucination, and it's the thing judges will test by asking odd questions live.

python

```
SYSTEM_PROMPT = """You are a research assistant with access to live web and news s

Rules:
1. Always use at least one tool before answering factual questions — never answer
2. Only state facts that appear in tool results. If tool results don't answer the
3. When you give your final answer, cite the specific source URL for each claim.
4. If the question has multiple angles (e.g. price + context), consider calling mo
5. Be concise. Do not repeat raw search results — synthesize them into a clear ans
"""
```

Step 4 — The agent loop (`src/agent.py`)

> **Concept — this is the actual "agent" part.** Everything before this was just tools. This loop is what makes it agentic: the model decides autonomously whether and which tool to call, potentially multiple times, before answering.

python

```python
import anthropic
from .tools import TOOL_SCHEMAS, TOOL_FUNCTIONS
from .prompts import SYSTEM_PROMPT

client = anthropic.Anthropic()

def run_agent(user_query: str, max_turns: int = 5) -> str:
    messages = [{"role": "user", "content": user_query}]

    for _ in range(max_turns):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            system=SYSTEM_PROMPT,
            tools=TOOL_SCHEMAS,
            messages=messages,
        )

        if response.stop_reason != "tool_use":
            # Final answer — extract and return
            return "".join(block.text for block in response.content if block.type

        # Model wants to call a tool
        messages.append({"role": "assistant", "content": response.content})
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                print(f"[agent] calling tool: {block.name}({block.input})")  # vis
                result = TOOL_FUNCTIONS[block.name](**block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": str(result),
                })
        messages.append({"role": "user", "content": tool_results})

    return "Reached max turns without a final answer."
```

Step 5 — CLI entry point (`src/main.py`)

```python
from dotenv import load_dotenv
load_dotenv()
from agent import run_agent

if __name__ == "__main__":
    query = input("Ask a question: ")
    print("\n" + run_agent(query))
```

---

6. Testing before recording your demo

Run 3–4 varied queries and watch the printed tool calls — this log output is genuinely useful to *show* in your demo video, since it proves the agent is really deciding, not hardcoded:

- A pure web-search question ("What is SerpApi used for?")
- A news-dependent question ("Any recent news about the RBI repo rate?")
- A question needing both tools ("What's the latest on iPhone prices in India and any news driving it?")
- An edge case with no good answer, to show the agent admits it rather than hallucinating

---

7. What separates a winning submission here

1. **Visible reasoning** — log/print each tool call so your demo video shows the "thinking," not just a final text blob.
2. **Grounded answers with real citations** — judges will likely test with a live question; if it hallucinates a source, that's an instant credibility loss.
3. **Clean README** — problem statement, architecture diagram (copy section 3 above), setup steps, and a 1-line note on which SerpApi engines/tools you used and why.
4. **Short demo video (2–3 min)**: show 2 example queries live, not slides.
5. **Public repo** with `.env.example` (never commit real keys) and a working `requirements.txt`.

---

8. Stretch goals (only if Step 1–7 are solid and you have time left)

- Add a third engine (e.g., `google_shopping`) for price-comparison questions.
- Add a simple Streamlit/Gradio UI instead of CLI, for a more visual demo.

- Add response caching to avoid redundant SerpApi calls during your own testing (saves API quota).

Do these *after* the core loop works end-to-end — a working CLI agent beats a broken UI every time.